

JobRadar — High-Level Design (HLD)

Oct 7, 2026

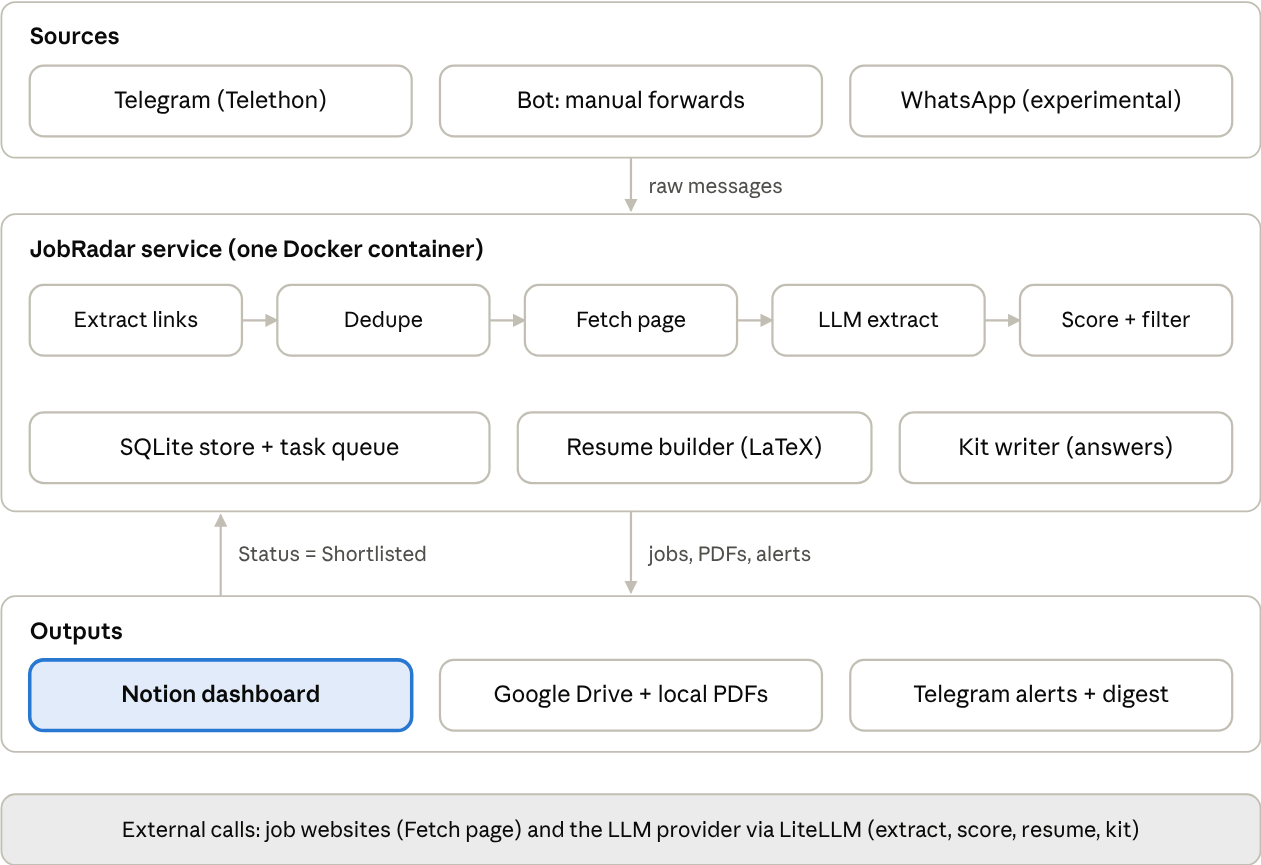
Summary and scope

JobRadar is a single self-hosted Python service built as an event-driven pipeline: sources push raw messages into a SQLite-backed queue, a chain of idempotent workers turns them into scored jobs, and sinks write results to Notion, Google Drive and a Telegram bot. Notion is both the dashboard and the control panel: a Status change there triggers resume generation.

This HLD covers v1.0 as defined in the PRD: Telegram ingestion, extraction, dedupe, scoring, LaTeX resumes, Notion dashboard, Drive storage and Telegram alerts. WhatsApp is an optional experimental source behind the same interface.

Architecture

Messages flow down through one service; Notion sends shortlists back up



architecture · sources, one service, outputs

Every pipeline step reads and writes the SQLite store through the task queue, so steps can retry independently. Notion is both an output and an input: the poller reads Status changes and queues resume and kit tasks.

Components and responsibilities

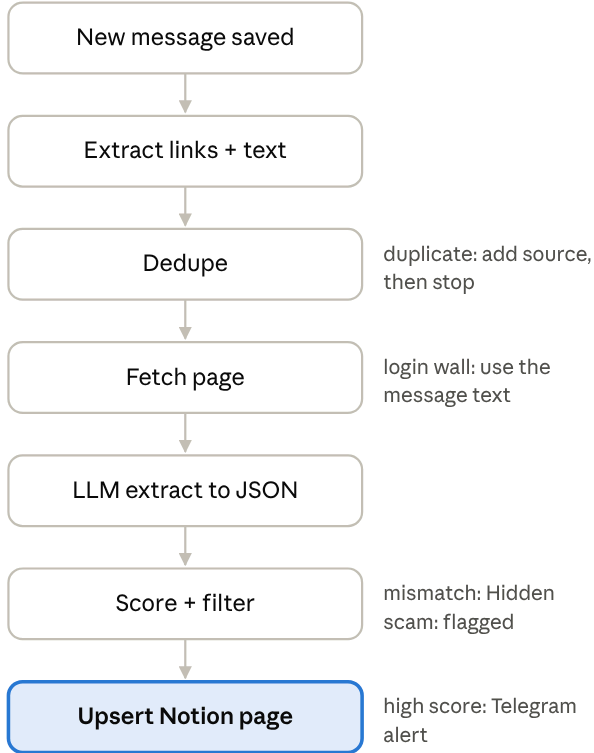
Component	Responsibility	Input → Output
Source adapters	Listen to Telegram (Telethon), the JobRadar bot (manual forwards) and optionally WhatsApp; normalise messages	Platform events → <code>RawMessage</code> rows

Component	Responsibility	Input → Output
Link extractor	Pull URLs from text, entities and buttons; expand short links; strip tracking params; OCR images	<code>RawMessage</code> → candidate URLs + text
Deduper	Canonical-URL hash and company+role+location fingerprint; attach extra sources to existing jobs	Candidates → new <code>Job</code> or merge
Fetcher	Download page with httpx; fall back to Playwright for JavaScript pages; skip login walls	URL → cleaned page text
Extractor (LLM)	Turn page and message text into the <code>JobPosting</code> JSON schema	Text → structured job
Scorer	Hard filters from config, LLM match score with reason, scam heuristics	Job + profile → score, status New or Hidden
Resume builder	Pick and rephrase profile content, render Jinja2 LaTeX, compile with Tectonic, validate	Job + profile → PDF
Kit writer	Draft answers for form fields and a cover letter	Job + profile → answers
Storage	Save PDFs locally; upload to Google Drive	PDF → path + share link
Notion sync	Upsert job pages (write); poll for Status changes (read)	Job ↔ Notion page
Notifier	Telegram bot alerts and twice-daily digest	Events → messages
Scheduler and queue	Run workers, retries with backoff, periodic jobs (Notion poll, digest, cleanup)	Tasks → executions
Store	SQLite: messages, jobs, sources, tasks, artifacts	Shared state

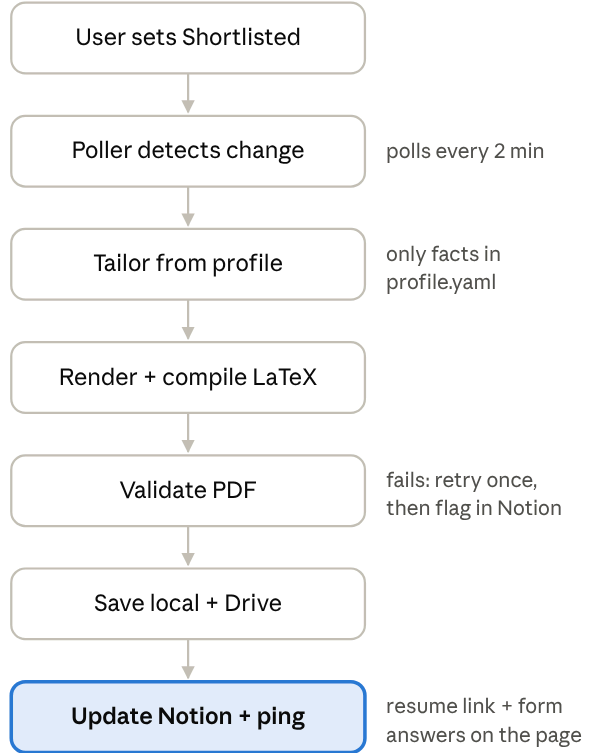
Key flows

Every job lands in Notion; a Shortlisted status sends it back for a resume

Flow A · new message to Notion



Flow B · Shortlisted to resume



key flows · ingestion and resume generation

Flow A runs for every message within about 2 minutes; Flow B runs only for jobs the user shortlists, which keeps LLM cost tied to real applications.

Data model overview

SQLite is the source of truth; Notion is a projection of it. Full column definitions are in the LLD.

Entity	Key fields	Notes
source	id, platform, chat_id, title, enabled	One per watched channel or group
raw_message	id, source_id, message_id, text, urls, received_at, processed	Unique on (source_id, message_id)

Entity	Key fields	Notes
<code>job</code>	id (hash), canonical_url, fingerprint, company, role, deadline, data_json, score, status, notion_page_id	One row per real job
<code>job_source</code>	job_id, raw_message_id	Every place a job was seen
<code>artifact</code>	id, job_id, kind (resume, kit), local_path, drive_url, version	Resumes and kits per job
<code>task</code>	id, type, payload, status, attempts, run_after, last_error	Durable work queue

The user profile lives in `profile.yaml1`, not the database, so it can be edited and versioned by hand.

External integrations

Service	How	Auth	Limits to respect
Telegram (reading)	Telethon user client over MTProto	api_id + api_hash from my.telegram.org, session file	FloodWait errors; read-only, no auto-join
Telegram (bot)	Bot API via python-telegram-bot	Bot token from BotFather	About 30 messages/second globally; plenty for alerts
WhatsApp (experimental)	Node sidecar with whatsapp-web.js or Baileys, posts messages to JobRadar over local HTTP	QR login	Unofficial; ban risk; off by default
LLM	LiteLLM (Claude, OpenAI, Gemini, Groq, Ollama)	Provider API key, or none for Ollama	Provider rate limits; cache by job id

Service	How	Auth	Limits to respect
Notion	Official API (notion-client)	Internal integration token, database shared with it	About 3 requests/second; 2,000 characters per text item; 100 blocks per append
Google Drive	Drive API v3	OAuth desktop flow, token stored locally	Per-user quota, ample for PDFs
Job websites	httpx, Playwright (Chromium)	None	1 request/second per domain; obey robots.txt; skip login walls

Tech stack

Layer	Choice	Why
Language	Python 3.11+, asyncio	Best libraries for Telegram, scraping and LLMs
Telegram	Telethon, python-telegram-bot	Mature, async
HTTP / browser	httpx, Playwright, trafilatura (main-text extraction)	Fast path plus a JavaScript fallback
OCR	Tesseract via pytesseract (optional: vision LLM)	Free, offline
LLM	LiteLLM + Pydantic schemas (instructor)	Any provider, validated JSON
Templates	Jinja2 with LaTeX-safe delimiters	Keeps LaTeX braces readable
LaTeX	Tectonic	Single binary, downloads packages on demand, small Docker image
Storage	SQLite (WAL mode) + SQLAlchemy	Zero setup, enough for one user
Config	pydantic-settings, YAML + <code>.env</code>	Typed and validated at start
Dashboard	Notion API (notion-client)	No frontend to build or host

Layer	Choice	Why
Packaging	Docker + docker compose, <code>uv</code> for dependencies	One-command install
Quality	pytest, ruff, mypy, GitHub Actions	Standard for open source

Deployment

One `docker compose up -d` starts everything on a laptop, a home server or a small VPS (1 vCPU, 1 GB RAM is enough without a local LLM).

Container	Contents	Required
<code>jobradar</code>	Python app, all workers, Tectonic, Tesseract	Yes
<code>browser</code>	Playwright Chromium (or bundled in <code>jobradar</code>)	Yes, for JavaScript pages
<code>wa-bridge</code>	Node WhatsApp sidecar	No, experimental profile
<code>ollama</code>	Local LLM	No, for free/private mode

Volumes: `./data` (SQLite, session files, Drive token), `./output` (PDFs), `./config` (config.yaml, profile.yaml, templates).

First run: `jobradar init` creates config from examples, logs in to Telegram (code sent to the app), runs the Google OAuth flow, and checks the Notion database schema.

Reliability, scaling and cost

Reliability

- Every message is written to SQLite before any processing, so a crash loses nothing.
- Each step is a durable `task` row; workers are idempotent (keyed by message id or job id), so retries are safe.
- Retries use exponential backoff (30 s, 2 min, 10 min, 1 h); after 5 failures the task is marked failed and shown in a Notion "Errors" view or log.
- On restart, Telethon catches up from the last stored message id per chat.

Scaling

- Target load (2,000 messages, 300 unique jobs a day) fits one process with about 4 concurrent fetch workers.
- Dedupe runs before any fetch or LLM call, so cost scales with unique jobs, not messages.
- Heavier users can swap SQLite for Postgres behind the same repository layer.

Cost control

- Cheap fast model for extraction and scoring; a stronger model only for resumes and answers.
- Resumes only for Shortlisted jobs (or above a score threshold).
- Cache LLM results by job id and prompt version; never re-extract an unchanged job.
- Daily LLM budget in config; when hit, extraction pauses and the digest says so.

Security and privacy

- **Secrets** (API keys, tokens) only in `.env`, loaded by pydantic-settings; never logged; `.env.example` ships with placeholders.
- **Telegram session file** gives full account access: stored in `./data` with permissions 600 and listed in `.gitignore`; docs warn never to share it.
- **Personal data** (`profile.yaml`, PDFs) stays local. Only job text and the profile fields needed for a task are sent to the LLM; a local Ollama model keeps everything on the machine.
- **Untrusted input**: job pages and messages may contain prompt-injection text. They are passed to the LLM only as data inside delimiters, outputs are schema-validated, and the LLM has no tools or write access.
- **LaTeX injection**: every inserted value is escaped; Tectonic runs with shell-escape disabled.
- **Outbound requests**: fetcher blocks private IP ranges and `file://` URLs to avoid SSRF from malicious links.
- **Repository hygiene**: gitleaks secret scanning in CI; Dependabot for dependency updates.

Extensibility (plugin model)

Four extension points, each a small Python interface registered by entry points (`jobradar.sources`, `jobradar.storage`, `jobradar.sinks`, `jobradar.site_adapters`). Contributors add a module and a line in `pyproject.toml`; the core does not change.

Extension point	Interface	Built-in implementations	Examples contributors could add
Source	<code>async def run(emit)</code>	telegram, bot_forward, whatsapp (experimental)	email inbox, RSS, Discord
Site adapter	<code>match(url)</code> , <code>extract(page)</code>	generic, google_forms	Greenhouse, Lever, Workday public pages
Storage	<code>save(pdf) -> ref</code>	local, gdrive	Dropbox, S3
Sink	<code>upsert(job)</code> , <code>poll_changes()</code>	notion, telegram_bot	Google Sheets, Airtable

LLM providers are pluggable through LiteLLM config, and resume templates are plain `.tex.j2` files in `templates/`.

Design decisions and alternatives

Decision	Chosen	Rejected alternative	Reason
Dashboard	Notion database	Custom React app, Streamlit	No frontend to build or host; duplicable template; works on phone
Telegram reading	User client (Telethon)	Bot API	Bots cannot read channels unless they are admins
Queue	SQLite task table	Redis + Celery	One user, one process; no extra service to run
Source of truth	SQLite, Notion as projection	Notion only	Notion is slow and rate-limited; local DB keeps dedupe fast and offline-safe

Decision	Chosen	Rejected alternative	Reason
Resume trigger	Status = Shortlisted in Notion	Generate for every job	Saves LLM cost and Drive clutter
LaTeX engine	Tectonic	TeX Live	About 50 MB vs several GB in the Docker image
LLM access	LiteLLM	One provider SDK	Users choose cost and privacy, including local models
Notion change detection	Polling every 2 minutes	Webhooks	Webhooks need a public URL; most users run at home
Applying	Human submits	Auto-fill and submit	Safer, honest, avoids bans and wrong submissions